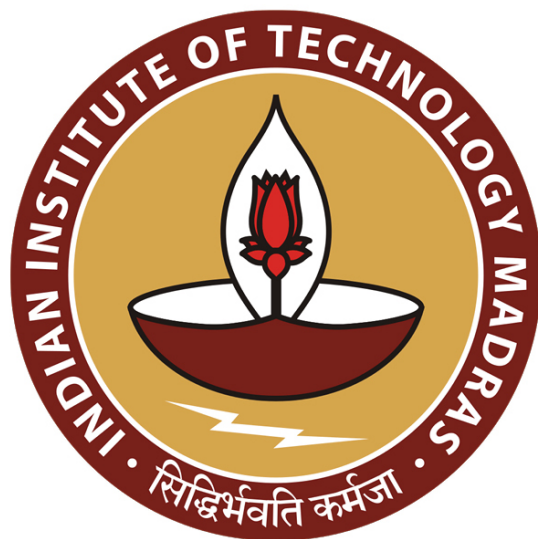


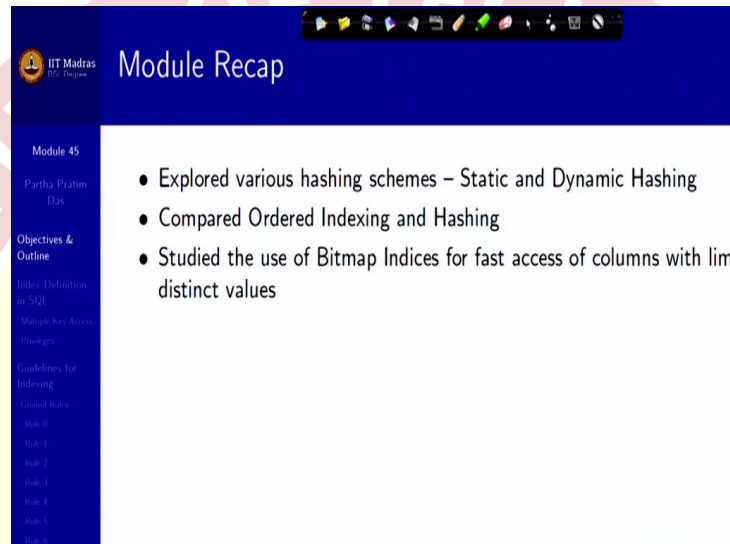


IIT Madras
ONLINE DEGREE

Database Management Systems
Professor Pratha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology, Kharagpur
Indexing and Hashing/5: Index Design

Welcome to module 45 of Database Management Systems course in the IIT Madras online B.Sc. program.

(Refer Slide Time: 0:30)

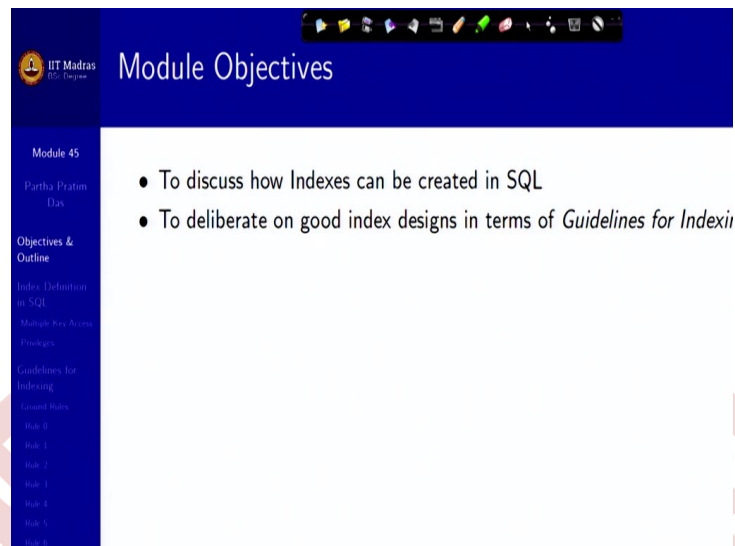


The screenshot shows a presentation slide titled "Module Recap" for "Module 45" of the IIT Madras online B.Sc. program. The slide is part of a course by Professor Pratha Pratim Das. The left sidebar lists the following topics: Module 45, Objectives & Outline, Index Distribution in SQL, Multiple Access Methods, Guidelines for Indexing, General Rules, Rule 0, Rule 1, Rule 2, Rule 3, Rule 4, Rule 5, and Rule 6. The main content area lists three bullet points:

- Explored various hashing schemes – Static and Dynamic Hashing
- Compared Ordered Indexing and Hashing
- Studied the use of Bitmap Indices for fast access of columns with limited distinct values

We have been discussing about indexing and hashing to make the accesses faster. And in the last week, we talked extensively about hashing particularly the static and dynamic hashing schemes and we compared the ordered indexing with hashing and we also took a look at a nice way of representing or making sure that you have fast access of columns, which has limited number of distinct values using a reverse mapping called the bitmap index.

(Refer Slide Time: 1:05)



IIT Madras
For Engineers

Module Objectives

Module 45
Partha Pratim Das

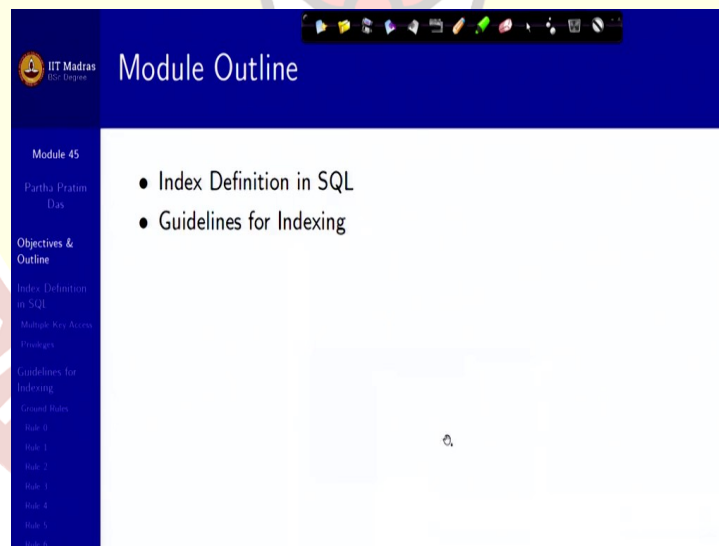
- To discuss how Indexes can be created in SQL
- To deliberate on good index designs in terms of *Guidelines for Indexing*

Objectives & Outline

- Index Definition in SQL
- Multiple Key Access
- Privileges
- Guidelines for Indexing
- Grant and Revoke
- Quiz 1
- Quiz 2
- Quiz 3
- Quiz 4
- Quiz 5
- Quiz 6

To close on these notions of indexing, we will in this module take a look at how do you create index of various kinds in SQL and also discuss few guidelines for what you should index, how you should index and what you should not be doing.

(Refer Slide Time: 1:26)



IIT Madras
For Engineers

Module Outline

Module 45
Partha Pratim Das

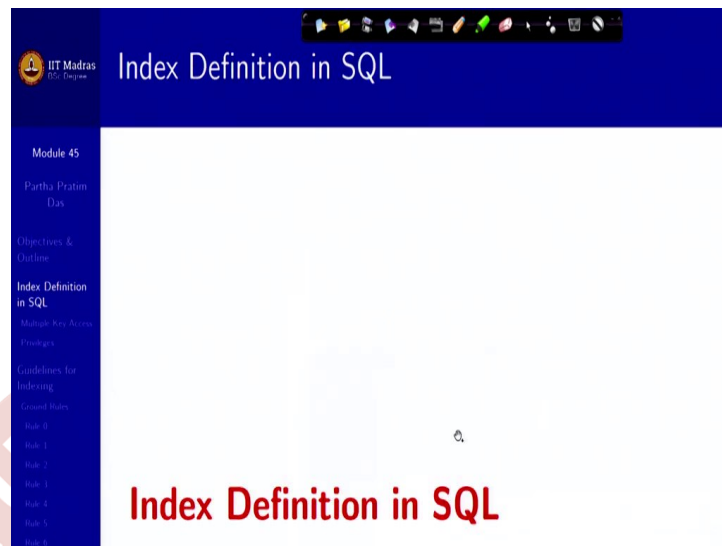
- Index Definition in SQL
- Guidelines for Indexing

Objectives & Outline

- Index Definition in SQL
- Multiple Key Access
- Privileges
- Guidelines for Indexing
- Grant and Revoke
- Quiz 1
- Quiz 2
- Quiz 3
- Quiz 4
- Quiz 5
- Quiz 6

So, index definition and guidelines.

(Refer Slide Time: 1:29)

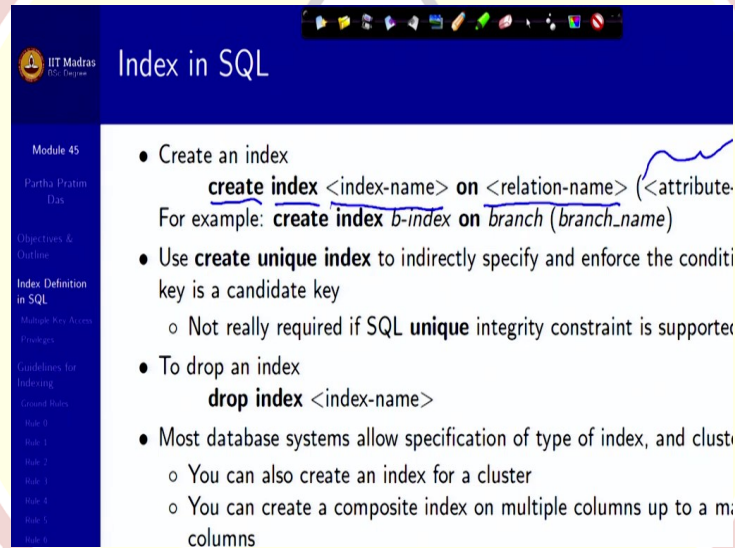


IIT Madras
B.Tech. Degree

Module 45
Partha Pratim Das

Objectives & Outline
Index Definition in SQL
Manage Key Access
Privileges
Guidelines for Indexing
Ground Rules
Rule 0
Rule 1
Rule 2
Rule 3
Rule 4
Rule 5
Rule 6

Index Definition in SQL



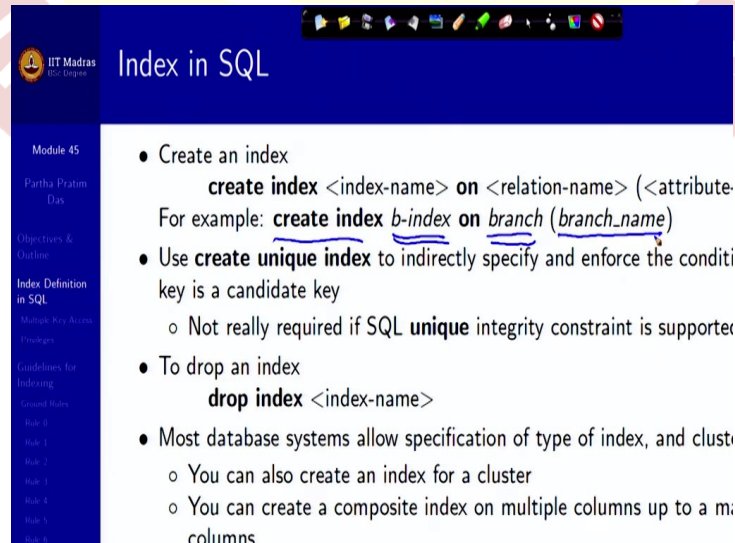
IIT Madras
B.Tech. Degree

Module 45
Partha Pratim Das

Objectives & Outline
Index Definition in SQL
Manage Key Access
Privileges
Guidelines for Indexing
Ground Rules
Rule 0
Rule 1
Rule 2
Rule 3
Rule 4
Rule 5
Rule 6

Index in SQL

- Create an index
create index <index-name> on <relation-name> (<attribute-name>)
For example: **create index b-index on branch (branch_name)**
- Use **create unique index** to indirectly specify and enforce the condition that the key is a candidate key
 - Not really required if SQL **unique** integrity constraint is supported
- To drop an index
drop index <index-name>
- Most database systems allow specification of type of index, and clustering
 - You can also create an index for a cluster
 - You can create a composite index on multiple columns up to a maximum of 16 columns



IIT Madras
B.Tech. Degree

Module 45
Partha Pratim Das

Objectives & Outline
Index Definition in SQL
Manage Key Access
Privileges
Guidelines for Indexing
Ground Rules
Rule 0
Rule 1
Rule 2
Rule 3
Rule 4
Rule 5
Rule 6

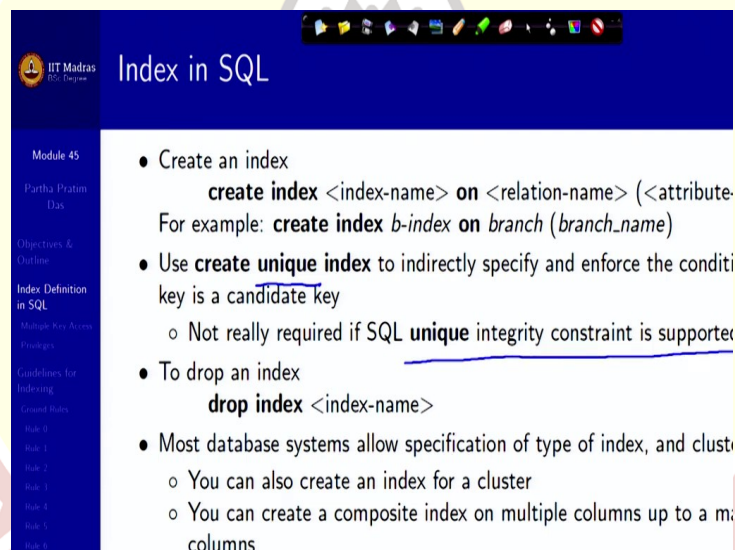
Index in SQL

- Create an index
create index <index-name> on <relation-name> (<attribute-name>)
For example: **create index b-index on branch (branch_name)**
- Use **create unique index** to indirectly specify and enforce the condition that the key is a candidate key
 - Not really required if SQL **unique** integrity constraint is supported
- To drop an index
drop index <index-name>
- Most database systems allow specification of type of index, and clustering
 - You can also create an index for a cluster
 - You can create a composite index on multiple columns up to a maximum of 16 columns

So, we start with the index definition in SQL. So, to create an index in SQL, it is I mean SQL style you have understood by now. So, if you have to create anything you have create, then you say what you are creating that is index give a name to that index, and then you have to specify which on which file on which relation are you creating it. So, you give a name of that relation.

And finally, is specify the list of attributes which would be used in index creating this index, it could be one attribute or more than one attribute that is an index could be simple or it could be composite, and we will come to the different issues of that, but this is the basic structure. So, the example as a example says create index this b-index is a name and we are creating index on a relational file called branch and using branch name as a attribute to be used in the index.

(Refer Slide Time: 2:30)



The screenshot shows a presentation slide titled "Index in SQL" from IIT Madras. The slide content is as follows:

- Create an index
create index <index-name> **on** <relation-name> (<attribute>
For example: **create index** *b-index* **on** *branch* (*branch_name*)
- Use **create unique index** to indirectly specify and enforce the condition that the key is a candidate key
 - Not really required if SQL **unique** integrity constraint is supported
- To drop an index
drop index <index-name>
- Most database systems allow specification of type of index, and clustering
 - You can also create an index for a cluster
 - You can create a composite index on multiple columns up to a maximum of 16 columns

Index in SQL

- Create an index
create index <index-name> on <relation-name> (<attribute>)
For example: **create index** *b-index* on *branch* (*branch_name*)
- Use **create unique index** to indirectly specify and enforce the condition that a search key is a candidate key
 - Not really required if SQL **unique** integrity constraint is supported
- To drop an index
drop index <index-name>
- Most database systems allow specification of type of index, and clustering
 - You can also create an index for a cluster
 - You can create a composite index on multiple columns up to a maximum of 32 columns

Index in SQL

- Create an index
create index <index-name> on <relation-name> (<attribute>)
For example: **create index** *b-index* on *branch* (*branch_name*)
- Use **create unique index** to indirectly specify and enforce the condition that a search key is a candidate key
 - Not really required if SQL **unique** integrity constraint is supported
- To drop an index
drop index <index-name>
- Most database systems allow specification of type of index, and clustering
 - You can also create an index for a cluster
 - You can create a composite index on multiple columns up to a maximum of 32 columns

You can also specify unique while writing the index to indirectly specify and enforce that the search key is a candidate key because if it becomes unique, it means that there cannot be duplicates of that value in the field. You it may not be required in SQL, if the unique integrity constraint is supported, if it is supported use that in your table design and do not have to use it in the index. But if it is not there, then you can also handle it for a specific index like this.

To delete an index or drop an index, it is called drop index like you have drop table and give the index name. So, this is a basic two commands to create and drop index. It also the index also optionally allow you to specify the type of index and the type of clustering you could create an index for a cluster, you could create composite index on multiple columns, maximum limit supported in SQL is 32 columns.

Of course, you would never in real designs you will hardly require that. So, the Composite Index key since you can have multiple columns involved will become large. So, there is a restriction that it cannot exceed roughly one-half of a data block of course, there is some more I mean, it has to be a little less than that, because some overhead data will also have to go in there, because otherwise, if it is more than half then you cannot keep the other information.

(Refer Slide Time: 4:21)

Index in SQL: Examples

- Create an index for a single column, to speed up queries that test that column:
 - `CREATE INDEX emp_ename ON emp.tab(ename);`
- Specify several storage settings explicitly for the index:
 - `CREATE INDEX emp_ename ON emp.tab(ename)`
 - `TABLESPACE users` // Allocation of space in the Database to contain
 - `STORAGE (` // Specify how Database should store a database object
 - `INITIAL 20K` // Specify the size of the 1st extent of the object
 - `NEXT 20K` // Specify in bytes the size of the 2nd extent to be al
 - `PCTINCREASE 75` // Specify the percent by which later extents
 - `PCTFREE 0` // 0% of each data block in this table's data segme
 - `COMPUTE STATISTICS;`
- Create index on two columns, to speed up queries that test either the first col
 - ▷ `CREATE INDEX emp_ename ON emp.tab(ename, empno) COMPUTE ST.`
- If a query is going to sort on the function UPPER(ENAME), an index on the f would not speed up this operation, and it might be slow to call the function fo

So, this is a basic way so to take more examples and showing you the options. So, you are creating an index, emp_ename ON employee table using the ename attribute as your indexing field. So, this will just create the index with default options, but you can specify a number of options, some key ones I have mentioned here.

One is you can specify the TABLESPACE. We did not talk about TABLESPACE. TABLESPACE is basically when the database is organizing its physical space. It has it might partition that into certain TABLESPACES. So, you can specify in which TABLESPACE you want that or otherwise, if you do not specify it will go to the default. So, this is a allocation of space in the database, which will contain the schema object, the index in this case.

Then you can specify the STORAGE options. So, this specifies how the database should store the option for example, how much should it allocate initially for this index schema object and how should that space grow and so, on. So, the options are that you can specify an INITIAL size that is called the first extent of the object for example, I have specifies 20 kilobytes and then you can specify the NEXT which is the number of bytes that by which it will be extended when once the initial extent is exhausted and you need to, need to have more storage. So, NEXT is used for the second extent.

And further on you do not specify the extent, but what you specify is a percentage by which every time it will increase over the last extent. So, here for example, we had 20k, then 20k So, the second extent was 100 percent increase, but from there on I am saying that the percentage increase will be 75. So, you have after these two you have 40k. So, 75 percent of that will be increased three-fourth of that which is so, it will become 30, increase would be 30k. So, it will become 70k. The fourth extent will do increase by 75 percent of 70k and so on.

So, so, you can see that, by percentage, the advantage you get is naturally as the index is growing, you would like to allocate more and more space in the subsequent extents, because it has a tendency to grow. If you feel that no it is initially only there is a lot of things will be required, but if the growth will be very low, then you could put PCTINCREASE percentage increase as a smaller value like say 10 percent, 20 percent.

You can also specify how much what percentage of the data block in the tables data segment should be kept free for updates, I have said the no nothing needs to be kept free. So, this is so, what we saw here from this part is the storage specification, then the PCTFREE and finally, you say which is again optional is compute statistics. So, that will tell you, how your index is doing, what is getting indexed, how often and so on so forth.

(Refer Slide Time: 8:03)



IIT Madras
For Everyone

Index in SQL: Examples

Module 45

Partha Pratim Das

Objectives & Outline

Index Definition in SQL

Managing User Access

Privileges

Guidelines for Indexing

General Rules

Rule 0

Rule 1

Rule 2

Rule 3

Rule 4

Rule 5

Rule 6

- Create an index for a single column, to speed up queries that test that column:
 - `CREATE INDEX emp_ename ON emp.tab(ename);`
- Specify several storage settings explicitly for the index:
 - `CREATE INDEX emp_ename ON emp.tab(ename)`
`TABLESPACE users` // Allocation of space in the Database to contain
`STORAGE (` // Specify how Database should store a database object
`INITIAL 20K` // Specify the size of the 1st extent of the object
`NEXT 20K` // Specify in bytes the size of the 2nd extent to be allocated
`PCTINCREASE 75)` // Specify the percent by which later extents can be increased
`PCTFREE 0` // 0% of each data block in this table's data segment should be free
`COMPUTE STATISTICS;`
- Create index on two columns, to speed up queries that test either the first column or the second column:
 - ▷ `CREATE INDEX emp_ename ON emp.tab(ename, empno) COMPUTE STATISTICS;`
- If a query is going to sort on the function `UPPER(ENAME)`, an index on the `ENAME` would not speed up this operation, and it might be slow to call the function for each row.



IIT Madras
For Everyone

Index in SQL: Examples

Module 45

Partha Pratim Das

Objectives & Outline

Index Definition in SQL

Managing User Access

Privileges

Guidelines for Indexing

General Rules

Rule 0

Rule 1

Rule 2

Rule 3

Rule 4

Rule 5

Rule 6

- Create an index for a single column, to speed up queries that test that column:
 - `CREATE INDEX emp_ename ON emp.tab(ename);`
- Specify several storage settings explicitly for the index:
 - `CREATE INDEX emp_ename ON emp.tab(ename)`
`TABLESPACE users` // Allocation of space in the Database to contain
`STORAGE (` // Specify how Database should store a database object
`INITIAL 20K` // Specify the size of the 1st extent of the object
`NEXT 20K` // Specify in bytes the size of the 2nd extent to be allocated
`PCTINCREASE 75)` // Specify the percent by which later extents can be increased
`PCTFREE 0` // 0% of each data block in this table's data segment should be free
`COMPUTE STATISTICS;`
- Create index on two columns, to speed up queries that test either the first column or the second column:
 - ▷ `CREATE INDEX emp_ename ON emp.tab(ename, empno) COMPUTE STATISTICS;`
- If a query is going to sort on the function `UPPER(ENAME)`, an index on the `ENAME` would not speed up this operation, and it might be slow to call the function for each row.



IIT Madras
For Everyone

Index in SQL: Examples

Module 45

Partha Pratim Das

Objectives & Outline

Index Definition in SQL

Managing User Access

Privileges

Guidelines for Indexing

General Rules

Rule 0

Rule 1

Rule 2

Rule 3

Rule 4

Rule 5

Rule 6

- Create an index for a single column, to speed up queries that test that column:
 - `CREATE INDEX emp_ename ON emp.tab(ename);`
- Specify several storage settings explicitly for the index:
 - `CREATE INDEX emp_ename ON emp.tab(ename)`
`TABLESPACE users` // Allocation of space in the Database to contain
`STORAGE (` // Specify how Database should store a database object
`INITIAL 20K` // Specify the size of the 1st extent of the object
`NEXT 20K` // Specify in bytes the size of the 2nd extent to be allocated
`PCTINCREASE 75)` // Specify the percent by which later extents can be increased
`PCTFREE 0` // 0% of each data block in this table's data segment should be free
`COMPUTE STATISTICS;`
- Create index on two columns, to speed up queries that test either the first column or the second column:
 - ▷ `CREATE INDEX emp_ename ON emp.tab(ename, empno) COMPUTE STATISTICS;`
- If a query is going to sort on the function `UPPER(ENAME)`, an index on the `ENAME` would not speed up this operation, and it might be slow to call the function for each row.

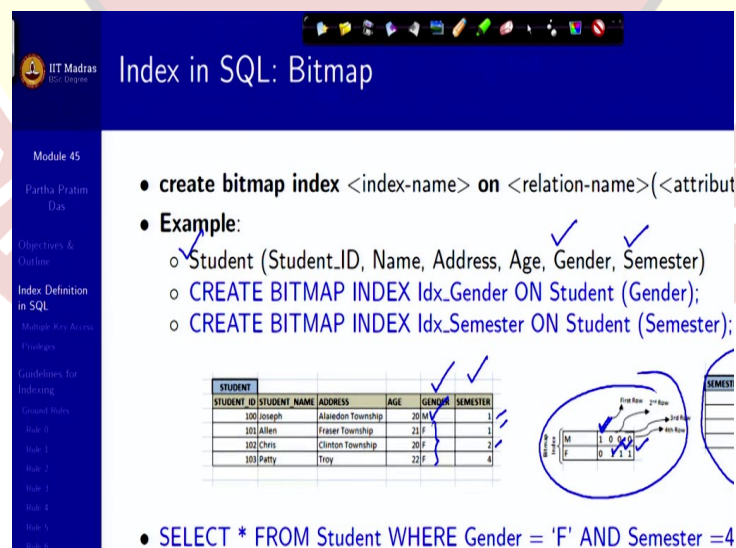
Now, to create index on two columns, you use the same command, but write two columns one after the other in the as their indexing attribute. So, there are I mean if you use two columns naturally the question is what will be used primarily and what will be used secondarily, we will talk about all that.

Now, if suppose a query is going to sort on the function or on the name, so you say, upper name so the an index on the ename column itself would not speed up this operation, because it has to sort, because sorting is, is changing the order. And the moment you change the order naturally, the index goes bad, it needs to be re computed, actually, it might slow down the call to this function.

So, if you want to have this kind of sorting be applied along with the indexing, then instead of creating the odd it on ename, you will have to create it with the function for each column value. So, that will speed up queries that use the function for searching or sorting, so you will have to keep this in mind.

So, these are the couple of basic things we have learned how to create an index, how to give very key, parameters to it if you want to control the space better, how to create index on multiple attributes and how to create index when a function is intended to be used on the column on which you are indexing.

(Refer Slide Time: 9:46)



Index in SQL: Bitmap

- **create bitmap index** <index-name> **on** <relation-name>(<attribute>)
- **Example:**
 - Student (Student_ID, Name, Address, Age, Gender, Semester)
 - **CREATE BITMAP INDEX** Idx_Gender **ON** Student (Gender);
 - **CREATE BITMAP INDEX** Idx_Semester **ON** Student (Semester);

STUDENT_ID	STUDENT_NAME	ADDRESS	AGE	GENDER	SEMESTER
100	Joseph	Alameda Township	20	M	1
101	Jalen	Fraser Township	21	F	1
102	Chris	Clinton Township	20	F	2
103	Patty	Troy	22	F	4

Diagram illustrating the bitmap index structure for Gender and Semester. The diagram shows a grid where rows represent data records and columns represent index values (M, F for Gender; 1, 2, 4 for Semester). Blue arrows indicate the mapping from data rows to index bits. For example, the first row (100) has M in the Gender column and 1 in the Semester column, corresponding to the first bit in the first row of the index grid.

- **SELECT * FROM Student WHERE Gender = 'F' AND Semester = 4**



Module 45

Partha Pratim Das

Objectives & Outline

Index Definition in SQL

Managing Access

Guidelines for Indexing

Current Data

Index 1

Index 2

Index 3

Index 4

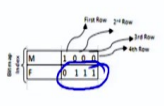
Index 5

Index 6

Index in SQL: Bitmap

- **create bitmap index** <index-name> **on** <relation-name>(<attribut
- **Example:**
 - Student (Student_ID, Name, Address, Age, Gender, Semester)
 - CREATE BITMAP INDEX Idx_Gender ON Student (Gender);
 - CREATE BITMAP INDEX Idx_Semester ON Student (Semester);

STUDENT_ID	STUDENT_NAME	ADDRESS	AGE	GENDER	SEMESTER
100	Joseph	Alaredon Township	20	M	1
101	Allen	Fraser Township	21	F	1
102	Chris	Clinton Township	20	F	2
103	Patty	They	22	F	4



- **SELECT * FROM Student WHERE Gender = 'F' AND Semester = 4**

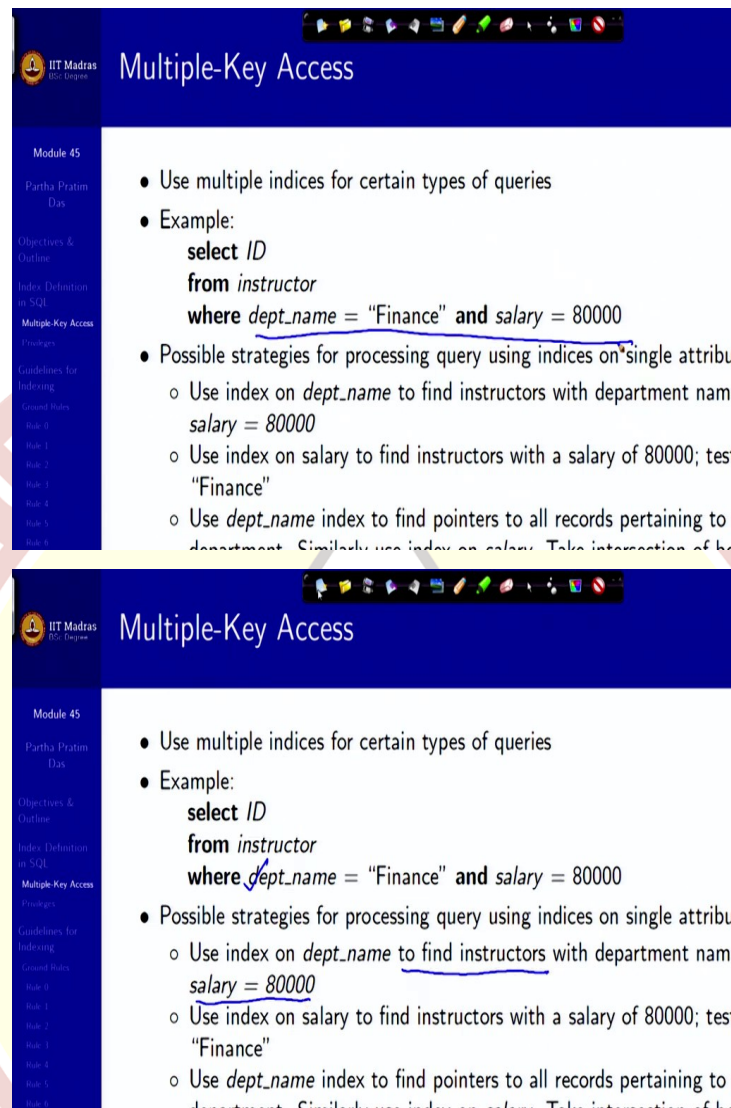
So, you can also create index using the bitmap we talked about bitmaps in hashing. So, here is an example. This standard student database ID, name, age, address, etc. And we know that these two are attributes which will have limited values, because there are 2 possible gender values and 4 possible semesters as we were looking at.

So, what we will do is for these two, we will use a bitmap index, which is as we said that it is a long bit array, as long as the number of records that exist in the table, and if a particular attribute is available in a record, then you put 1, otherwise you put 0. So, first record has, 101 has gender as M, so, it is 1 here and the rest are f so, you put 1 in those.

Similarly, you can, you do the same thing on semester because there are 4 semesters and you will have for the first and second you have semester 1, for the third, for the third record, you have semester 2, for semester 3, you have no record and the last one. So, with this your several of your computation like you want to do select star from where gender f and semester gender is equal to F and semester is equal to 4.

So, what do you take you take the bit pattern of f, you take the bit pattern of semester 4, and all that you need to do is to do an ending. So, take these and take these do an ending so you will get 0001 which tells you that the fourth record, fourth record only satisfies this condition. So, this naturally, this kind of index speeds up operations when you have this kind of a property. So, all that you need to do is to say that specify the bitmap field on which bitmap keyword to create a bitmap index and rest of it is the same.

(Refer Slide Time: 12:09)



Multiple-Key Access

Module 45
Partha Pratim Das
Objectives & Outline
Index Definition in SQL
Multiple-Key Access
Prologues
Guidelines for Indexing
General Rules
Rule 1
Rule 2
Rule 3
Rule 4
Rule 5
Rule 6

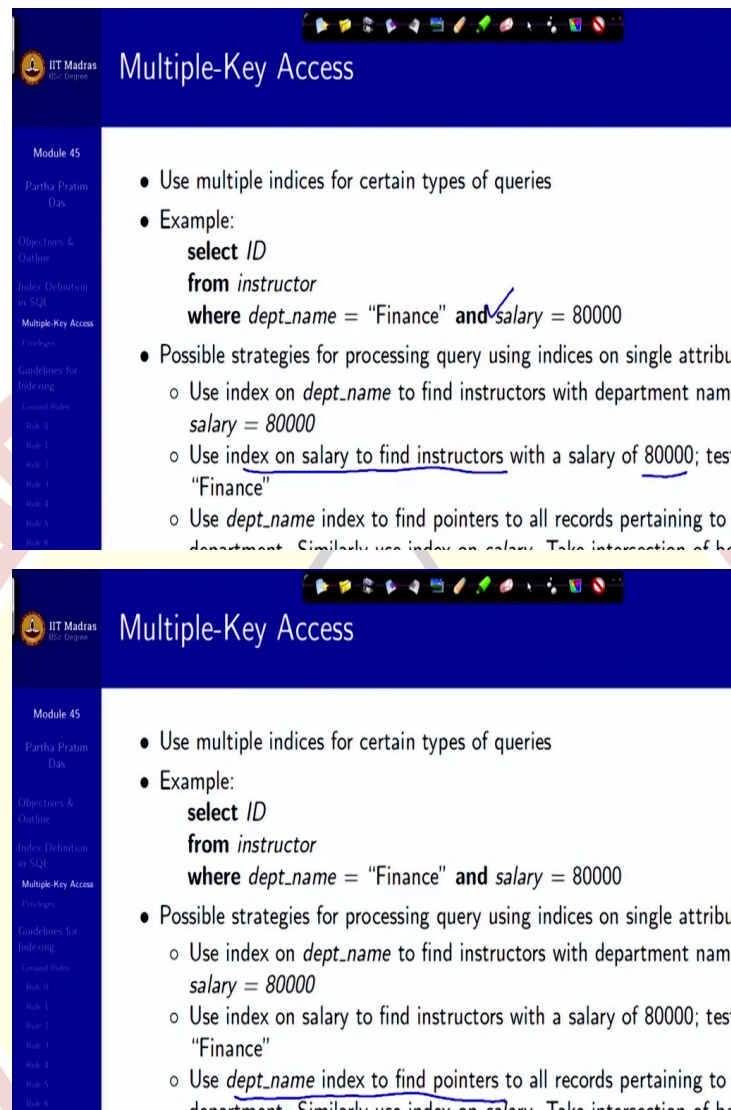
- Use multiple indices for certain types of queries
- Example:

```
select ID
from instructor
where dept_name = "Finance" and salary = 80000
```
- Possible strategies for processing query using indices on single attribute
 - Use index on *dept_name* to find instructors with department name "Finance"
 - Use index on *salary* to find instructors with a salary of 80000; test "Finance"
 - Use *dept_name* index to find pointers to all records pertaining to department. Similarly use index on *salary*. Take intersection of both.

Now, you can have multi key access that is what because there could be more than one key, more than one attributes on which you want to index because oftentimes, maybe you use them in a combined manner like you are doing it here that select ID of the instructor where the department name is finance and salary is 80000. So, what could be the possible strategy?

One possible strategy is to use an index on the department name only, I mean just use the index on the department name. So, that will quickly find new instructors with that given department name finance, and then you do a normal test for the salary because you have index on this.

(Refer Slide Time: 12:59)



IIT Madras
Multiple-Key Access

Module 45
Partha Pratim Das
Objectives & Outline
Index Definition in SQL
Multiple-Key Access
Prologues
Guidelines for Indexing
General Rules
Rule 1
Rule 2
Rule 3
Rule 4
Rule 5
Rule 6

- Use multiple indices for certain types of queries
- Example:

```
select ID
from instructor
where dept_name = "Finance" and salary = 80000
```
- Possible strategies for processing query using indices on single attributes
 - Use index on *dept_name* to find instructors with department name "Finance"
 - Use index on *salary* to find instructors with a salary of 80000; test for "Finance"
 - Use *dept_name* index to find pointers to all records pertaining to "Finance" department. Similarly use index on *salary*. Take intersection of both sets.

Alternately, you could have index on salary to find the instructors with salary 80,000 and then test for the department name or you can use department name to find, department name index to find pointers to all records pertaining to finance and similarly use index on salary. So, intersection of both sets are the set of pointers obtained. So, these are the different kinds of possibilities when you have multiple such.

(Refer Slide Time: 13:35)

Multiple-Key Access (2): Indices

Module 45
Partha Pratim Das
Objectives & Outline
Index Definition in SQL
Multiple-Key Access
Primitives
Guidelines for Indexing
General Rules
Rule 1
Rule 2
Rule 3
Rule 4
Rule 5
Rule 6

- **Composite Search Keys** are search keys containing more than one attribute.
 - For example, $(dept_name, salary)$
- Lexicographic ordering: $(a_1, a_2) < (b_1, b_2)$ if either
 - $a_1 < b_1$, or
 - $a_1 = b_1$ and $a_2 < b_2$
- Hence, the order is important

Multiple-Key Access (2): Indices

Module 45
Partha Pratim Das
Objectives & Outline
Index Definition in SQL
Multiple-Key Access
Primitives
Guidelines for Indexing
General Rules
Rule 1
Rule 2
Rule 3
Rule 4
Rule 5
Rule 6

- **Composite Search Keys** are search keys containing more than one attribute.
 - For example, $(dept_name, salary)$
- Lexicographic ordering: $(a_1, a_2) < (b_1, b_2)$ if either
 - $a_1 < b_1$, or
 - $a_1 = b_1$ and $a_2 < b_2$
- Hence, the order is important

Now, if you use this as a composite index a department name salary, then the question is in terms of indexing, indexing is basically creating a virtual ordering; indexing gives you a virtual sorting on the data. So, what is that sorting rule? Sorting rule is there is this order is important from left to right and the in this order whichever comes first is given the preference in terms of the comparison.

So, if I order in this way, then the ordering is $a_1 a_2$ is less than considered to be less than $b_1 b_2$ this tuple because you have two values now, if either a_1 is less than b_1 , so you do not look at the other one. So, you consider that to be the earlier record. Or if a_1 is equal to b_1 and a_2 is less than b_2 . So, this is the way you know you arrange the dictionary. So exactly you use that and therefore the order is important. So, the index order that will be created by an index,

which is department name, salary is different from what will be created if it is salary, department name.

(Refer Slide Time: 14:54)

Multiple-Key Access (3): Indices on Multiple Attributes

Module 45
Partha Pratim Das

Suppose we have an index on combined search-key (dept_name, salary)

- With the **where** clause
where dept_name = "Finance" and salary = 80000 ✓
the index on (dept_name, salary) can be used to fetch only records that satisfy both conditions.
 - Using separate indices is less efficient - we may fetch many records that satisfy only one of the conditions
 - Can also efficiently handle
where dept_name = "Finance" and salary < 80000
 - But cannot efficiently handle
where dept_name < "Finance" and salary = 80000

Multiple-Key Access (3): Indices on Multiple Attributes

Module 45
Partha Pratim Das

Suppose we have an index on combined search-key (dept_name, salary)

- With the **where** clause
where dept_name = "Finance" and salary = 80000
the index on (dept_name, salary) can be used to fetch only records that satisfy both conditions.
 - Using separate indices is less efficient - we may fetch many records that satisfy only one of the conditions
 - Can also efficiently handle
where dept_name = "Finance" and salary < 80000
 - But cannot efficiently handle
where dept_name < "Finance" and salary = 80000

So, when you index with this kind of multiple attributes, let us say you have done an index on department name, salary and you are trying to do this though. So, this can be used to fetch only records that satisfy both conditions. Now, if you use separate indices, it will be less efficient, because each one of them will bring more records than what will actually go in that final case.

Actually, this will also be efficient in terms of this query, the difference is salary is less, why it is so? Because in department and salary, the first preference is being given to department name. So, department name has the primary ordering, so, it will filter out lot more because it

has an equality, department name is finance. So, it will be it is expectedly, much less number of tuples within which the index, second part of the index on salary will be used to get records less than 80,000.

But, if my query was different, just think of it is department name is less than finance, whatever it means, I mean, it is not a semantically very meaningful, the meaningful query, but this is equal to 80,000, then this will bring a lot more number of records, because primarily it is going to look at the department name. So, not the second condition.

So, while you form your index, you have to also keep an eye on what kind of queries you are going to primarily look for or what kind of for which kind of queries do you need to do a speed up, you are intending to do a speed up using the, using all your indexing mechanisms.

(Refer Slide Time: 16:50)

Privileges Required to Create an Index

- When using indexes in an application, you might need to request that privileges or make changes to initialization parameters
- To create a new index
 - You must own, or have the INDEX object privilege for the corresponding schema
 - The schema that contains the index must also have a quota for the tablespace intended to contain the index, or the UNLIMITED TABLESPACE system privilege
 - To create an index in another user's schema, you must have the CREATE INDEX system privilege
- Function-based indexes also require the QUERY_REWRITE privilege, and the QUERY_REWRITE_ENABLED initialization parameter to be set to TRUE

Now moving on, can anybody create an index? The answer is no, because creating an index has a lot of impact in terms of speeding up a few queries, which is true, but it may also impact in terms of various performance degradation in updates, we will discuss, I mean, you can easily understand that because every update will have to update the index, you, it also has implications in terms of the space that you use in the database.

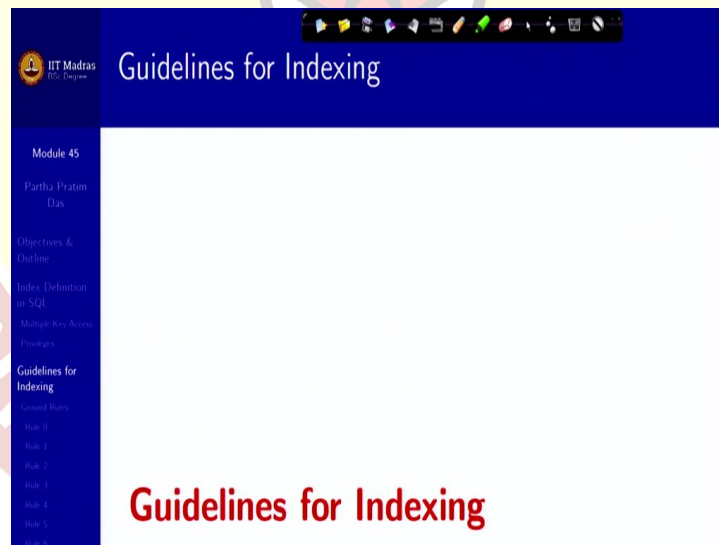
So, there are certain rights or privileges that you will need to have to be able to create an index otherwise you will have to, I mean for that you will have to ask your DBA to grant the privileges or otherwise you will have to just work with the indexes that exists. So, you must have a INDEX object privilege. And if the schema contains the index must also have a quota

for the TABLESPACE you can specify this much of TABLESPACE I need, then you will need to have that kind of a system privilege, unlimited TABLESPACE and so on.

If you want to create an index on a schema that is not owned by you, it is owned by someone else, then you must have CREATE ANY INDEX privilege. One is a create index privilege, one is create any by create index privilege, you can create index for schemas that you own. With create any index privilege, you can also create schema that index for schema that others own.

Similarly, function-based indexes, which we saw in terms of upper ename required the QUERY REWRITE privilege, or and the QUERY REWRITE ENABLED initialization parameter has to be set to true for that, I am sure you will not retain so much of details from this description. All that you need to remember is you need privileges and when you are actually working on the system, you can look up the system manual to see exactly what privileges you need and ask the DBA to grant you those privileges. DBA means database administrator; the ultimate authority.

(Refer Slide Time: 19:13)





Guidelines for Indexing

Module 45
Partha Pratim Das

- Objectives & Outline
- Index Definition in SQL
- Manage Key Access Privileges
- Guidelines for Indexing**
 - General Rules
 - Rule 0
 - Rule 1
 - Rule 2
 - Rule 3
 - Rule 4
 - Rule 5
 - Rule 6

- In Modules 16 to 20 (Week 4), we have studied various issues for a relational database system. This focused on:
 - Normalization of Tables** leading to
 - Reduction of Redundancy to minimize possibilities of Anomaly
 - Easier adherence to constraints (various dependencies)
 - Efficiency of access and update – a better normalized design c performance

So, this was how you create indexes in SQL. Now, the question is, what should you index and how much should you index? So, if we step back a little, you will see that there have been certain basic design objectives we started with. And initially it was totally at the design level so we work extensively on normalization of tables, which lead to reduction of redundancy naturally, consequently minimizes the possibilities of anomaly, it become, it makes it easier to adhere to different constraints by various dependencies and efficiency of access and update is better in a normalized design.

So, this is what you do at a design time, but remember design is done once, once you have, I mean, you hardly ever change the design because if you change the design in a live database, then all your data will have to be migrated and all those issues will come.

(Refer Slide Time: 20:16)



Guidelines for Indexing (2)

Module 45
Partha Pratim Das

- Objectives & Outline
- Index Definition in SQL
- Manage Key Access Privileges
- Guidelines for Indexing**
 - General Rules
 - Rule 0
 - Rule 1
 - Rule 2
 - Rule 3
 - Rule 4
 - Rule 5
 - Rule 6

- The performance of a database system, however, is also significantly way the data is physically organized and managed. These are done th
 - Indexing and Hashing**
- While normalization and design are startup time activities that are us once at the beginning (and rarely changed later), the performance be to evolve as the database is used over time. Hence we need to contin
 - Collect Statistics** about data (of various tables) to learn of the p
 - Adjust the Indexes** on the tables to optimize performance
- There is no sound theory that determines optimal performance. Rath look into a few common guidelines that can help you keep your data behavior

IIT Madras
Hyderabad

Guidelines for Indexing (2)

Module 45
Partha Pratim Das

- The performance of a database system, however, is also significantly way the data is physically organized and managed. These are done through
 - **Indexing and Hashing**
- While normalization and design are startup time activities that are used once at the beginning (and rarely changed later), the performance has to evolve as the database is used over time. Hence we need to continue
 - **Collect Statistics** about data (of various tables) to learn of the patterns
 - **Adjust the Indexes** on the tables to optimize performance
- There is no sound theory that determines optimal performance. Rather, we look into a few common guidelines that can help you keep your database behavior

So, subsequent your design what do you have to keep doing to have a good database performance is to see how the data is physically organized and managed. So, that is the next thing we have talked about extensively that is about indexing and hashing, what kind of index, what kind of hashes all these will have to be all these mechanisms have been given so that you can based on what your database needs at the execution time at the runtime with the real data depending on the idiosyncrasies of the real data that will have to be designed.

So, naturally How do you know about how your data is doing and these are the two tools that you have, one is collect statistics as we say compute statistics. So, which will give you a variety of on various tables, you will be able to learn the patterns which query is moving faster, which query is more frequent, what is slowing down a certain query processing or update processing and so on. And based on that, you have to adjust the indexes on the tables to optimize the performance.

Now, unfortunately, unlike the normalization theory, there is no sound theory that can determine optimal performance, because it significantly depends on the profile of the data in the real world. Given the same basic normalize design from the constraints of the data, you could have variety of different actual profiles of the data, statistics of the data requiring varied different kinds of indexes depending on the data distribution.

So, since we do not have a theory, what I would just show to you is a set of you know kind of rules, these rules are not, not any rules like programming language rules, these are cast in stone, these are kind of guidelines, which has evolved over a large period of experience that the community has gathered by working with databases. So, you should keep these rules in

your mind and then apply them as appropriate in different contexts based on the statistics and adjust the indices, so that your database remains agile in its behavior all the time.

(Refer Slide Time: 22:48)

Guidelines for Indexing: Ground Rules

- Some guidelines - heuristic and common sense, but time-tested - are as a set of Ground Rules for Indexing
 - **Rule 0:** *Indexes lead to Access - Update Tradeoff* ✓
 - **Rule 1:** *Index the Correct Tables* ✓
 - **Rule 2:** *Index the Correct Columns* ✓
 - **Rule 3:** *Limit the Number of Indexes for Each Table* ✓
 - **Rule 4:** *Choose the Order of Columns in Composite Indexes* ✓
 - **Rule 5:** *Gather Statistics to Make Index Usage More Accurate* ✓
 - **Rule 6:** *Drop Indexes That Are No Longer Required* ✓
- These rules are explained in the following slides

So, I summarize I start with the summary of 7 ground rules for indexing. Rule 0 indexes lead to access, update, trade-off the more index you have you have better access more index you have updates become slower, disk space becomes more. So, there is a trade-off you will have to do it is not that you can have a design by indexing everything, disastrous, but at the same time, if you do not index then anything, then your frequently asked queries will be unnecessarily slow.

The second you have to look at is, index the correct table not index everything, which table should you index you have to answer that, do you index every table? Or there are certain properties that you could look for, in a table what are the correct columns to index? Would you index on every column every combination that is explosive? So, you have to decide on the correct.

What is a number of different indexes that you will have on each table? So, this very, these are more significantly common-sense rules, but very significant in terms of actual execution. You may have composite indexes and the order of the columns we have explained with this department name and salary example that how the order of the columns become important in terms of the composite indexes. Then you have to gather statistics to make index usage more accurate and finally, drop the indexes that are no longer required. So, these are the 7 ground rules.

(Refer Slide Time: 24:39)

Guidelines for Indexing: Rule 0

- **Rule 0: Indexes lead to Access - Update Tradeoff**
 - Every query (access) results in a 'search' on the underlying physical data
 - ▷ Having specific index on search field can significantly improve performance
 - Every update (insert / delete / values update) results in update of the index file
 - ▷ Having unnecessary indexes can cause significant degradation of various operations
 - ▷ Index files may also occupy significant space on your disk and cause slow behavior due to memory limitations during index creation
 - Use informed judgment to index!

Rule 0 indexes lead to access update, trade off, so you know that the query access results in a search so index will help. So, specific index on that field or the set of fields will help and but update any update insert, delete or even a values update result in the update of the index file also. So, we have, for that reason only, we have seen all this self-adjusting B+ tree structure to maintain the data.

So, but it has a overhead or penalty. So, update is the overhead or penalty you pay for quicker access. So, having unnecessary indexes will cause significant degradation of performance. The index files could also occupy a lot of space. So, you have to have informed judgment to index, this is the basic first rule.

(Refer Slide Time: 25:43)

Guidelines for Indexing: Rule 1

- **Rule 1: Index the Correct Tables**
 - Create an index if you frequently want to retrieve less than 15% of a large table
 - ▷ The percentage varies greatly according to the relative speed of the table scan and how clustered the row data is about the index key
 - The faster the table scan, the lower the percentage
 - More clustered the row data, the higher the percentage
 - Index columns used for joins to improve performance on joins of multiple tables
 - Primary and unique keys automatically have indexes, but you might want an index on a foreign key
 - **Small tables** do not require indexes

Second says index the correct table. So, this is a thumb rule being given create an index if you freak, I am sorry, create an index if you frequently want to retrieve less than 15 percent of the rows in a large table. Why, why so? Now, this percentage is not sacrosanct, it will vary according to the relative speed of the table scan and how clustered the row data are, you can think of it in this way that if the data is more clustered, lot of values are together, then you will have the need to index only if you want to access a whole lot of data.

But if it is very sparse in terms of clustered lot of values are distinct, then even for small segment, small part of the data, small part of the rules being accessed, an index will help. The core idea behind this is index is good on distinct values. So, you have to make an assessment of how much of distinctness you have in your table in terms of those values.

Index columns can also be used to improve performance on join of multiple tables. So, you will have to know what you want to join. And remember, while indexing a table, that primary and unique keys are automatically indexed by the database, you do not have to create indexes for that. But you might want to create an index on a foreign key, which can make your joins better.

Last but not the least, if a table is small, you do not need to require an index, because it might actually have to do much of overhead, that serial scan in a small table would be fine. But then over a period of time, when you see that your query is taking too long, you know that your table have grown to a from a small to large size where you will have to index it. So, it is a live scenario and you will have to keep managing that.

(Refer Slide Time: 28:06)

Guidelines for Indexing: Rule 2

- **Rule 2: Index the Correct Columns**
 - Columns with the following characteristics are candidates for index:
 - ▷ Values are relatively unique in the column ✓
 - ▷ There is a wide range of values (good for regular indexes) ✓
 - ▷ There is a small range of values (good for bitmap indexes) ✓
 - ▷ The column contains many nulls, but queries often select all non-null value. In this case, a comparison that matches all the non-null value.
 - WHERE COL_X > -9.99 * power(10, 125) is preferable to IS NOT NULL
 - This is because the first uses an index on COL_X (if COL_X is indexed)
 - Columns with the following characteristics are less suitable for index:
 - There are many nulls in the column and you do not search on

Guidelines for Indexing: Rule 2

- **Rule 2: Index the Correct Columns**
 - Columns with the following characteristics are candidates for indexing:
 - ▷ Values are relatively unique in the column
 - ▷ There is a wide range of values (good for regular indexes)
 - ▷ There is a small range of values (good for bitmap indexes)
 - ▷ The column contains many nulls, but queries often select all the non-null value. In this case, a comparison that matches all the non-null values is preferable.
 - `WHERE COL_X > -9.99 * power(10, 125)` is preferable to `IS NOT NULL`
 - This is because the first uses an index on COL_X (if COL_X is indexed)
 - Columns with the following characteristics are less suitable for indexing:
 - ▷ There are many nulls in the column and most of the non-null values are...

Guidelines for Indexing: Rule 2

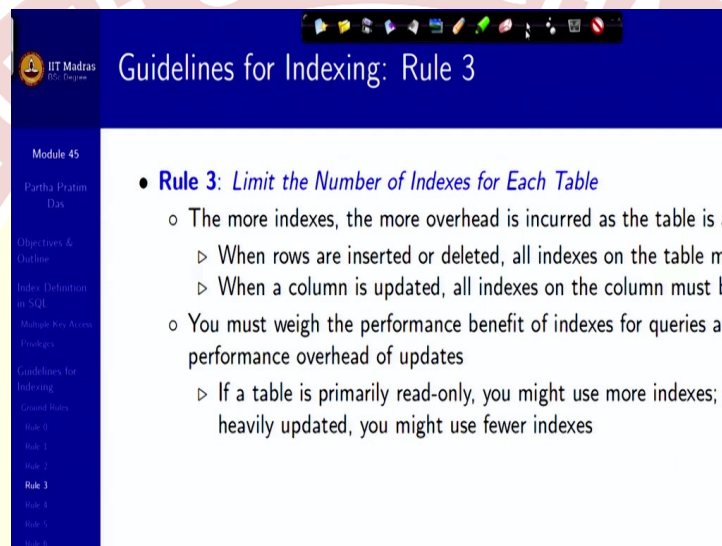
- **Rule 2: Index the Correct Columns**
 - Columns with the following characteristics are candidates for indexing:
 - ▷ Values are relatively unique in the column
 - ▷ There is a wide range of values (good for regular indexes)
 - ▷ There is a small range of values (good for bitmap indexes)
 - ▷ The column contains many nulls, but queries often select all the non-null value. In this case, a comparison that matches all the non-null values is preferable.
 - `WHERE COL_X > -9.99 * power(10, 125)` is preferable to `IS NOT NULL`
 - This is because the first uses an index on COL_X (if COL_X is indexed)
 - Columns with the following characteristics are less suitable for indexing:
 - ▷ There are many nulls in the column and most of the non-null values are...

Index the correct column. So, as I said that indexes work well when the values are relatively unique. And there is a wide range of values you may have use the regular index, you have a smaller range of values like gender, semester and so on seasons, use a bitmap index there is a kind of tape to handle columns containing many nulls, so, you will, if you want to deal with the non-null values, you will normally say that where column x is not null that is a direct semantic way of doing it.

So, instead of saying that, you could say something like this, if you understand, so, what this is saying is minus 9.99 times 10 to the power 125 which means that it is an extremely small negative value. So, we are saying column is greater than this. So, expectedly whatever value you have in that column, obviously, this is true for numerical numeric column not for others. For a numeric column, whatever value we have expectedly it will be larger than this except for those which are null, because those will never match in a comparison.

So, what you, why do you want to prefer this because if you use this, and you have an index on column x, then it will use that index but if you use this it cannot use the index because null cannot be indexed. So, there are, these are some of the tips there if there are, if there are many nulls in the column and you do not search on the non-null values, then you do not index long and long row columns cannot be indexed, of course, long row is kind of deprecated now, it is blob, a blob you have you have, I have talked about blob that is that the binary data. So, this is your rule number 2.

(Refer Slide Time: 30:34)

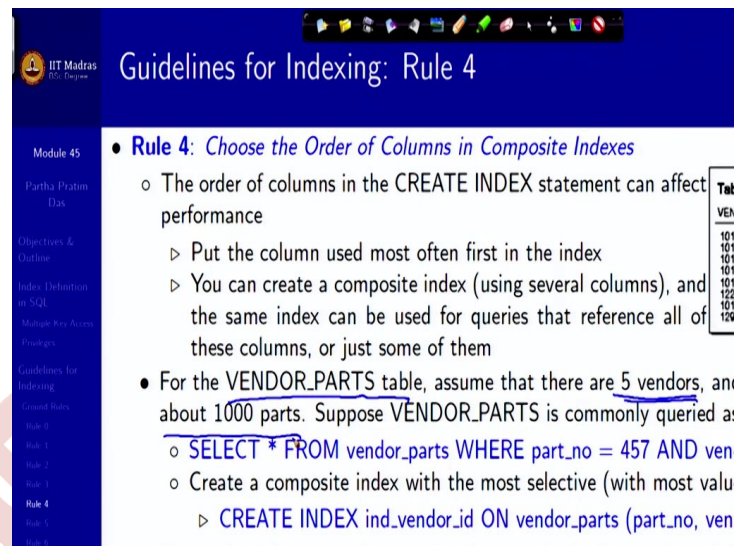


The screenshot shows a presentation slide from IIT Madras. The title is 'Guidelines for Indexing: Rule 3'. The slide content is as follows:

- **Rule 3: Limit the Number of Indexes for Each Table**
 - The more indexes, the more overhead is incurred as the table is altered
 - ▷ When rows are inserted or deleted, all indexes on the table must be updated
 - ▷ When a column is updated, all indexes on the column must be updated
 - You must weigh the performance benefit of indexes for queries against the performance overhead of updates
 - ▷ If a table is primarily read-only, you might use more indexes; but if the table is heavily updated, you might use fewer indexes

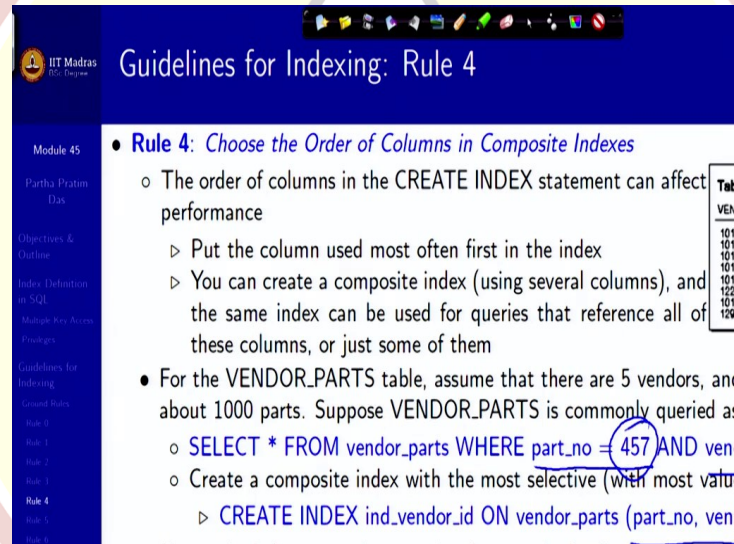
Rule number 3 says that the limit the number of indexes for each table, because if you have more indexes, moreover, it is incurred. So, when a row is inserted or deleted, all indexes will have to be updated. So, you must check that you really need the indexes that you have. And if a table is primarily read only, you might use more index because, then that possibility of data changes are very few. But if the table is heavily updated, you must use fewer indices.

(Refer Slide Time: 31:07)



Guidelines for Indexing: Rule 4

- **Rule 4: Choose the Order of Columns in Composite Indexes**
 - The order of columns in the CREATE INDEX statement can affect performance
 - ▷ Put the column used most often first in the index
 - ▷ You can create a composite index (using several columns), and the same index can be used for queries that reference all of these columns, or just some of them
 - For the VENDOR_PARTS table, assume that there are 5 vendors, and about 1000 parts. Suppose VENDOR_PARTS is commonly queried as:
 - `SELECT * FROM vendor_parts WHERE part_no = 457 AND vendor_id = 101`
 - Create a composite index with the most selective (with most values) first in the index
 - ▷ `CREATE INDEX ind_vendor_id ON vendor_parts (part_no, vendor_id)`



Guidelines for Indexing: Rule 4

- **Rule 4: Choose the Order of Columns in Composite Indexes**
 - The order of columns in the CREATE INDEX statement can affect performance
 - ▷ Put the column used most often first in the index
 - ▷ You can create a composite index (using several columns), and the same index can be used for queries that reference all of these columns, or just some of them
 - For the VENDOR_PARTS table, assume that there are 5 vendors, and about 1000 parts. Suppose VENDOR_PARTS is commonly queried as:
 - `SELECT * FROM vendor_parts WHERE part_no = 457 AND vendor_id = 101`
 - Create a composite index with the most selective (with most values) first in the index
 - ▷ `CREATE INDEX ind_vendor_id ON vendor_parts (part_no, vendor_id)`

Choose the order of columns in the composite Index, we have already talked about this, here is an example that I would like to, like you to look at, say there is a vendor table given here and it has vendor parts table, it has 5 vendors, and each vendor has about 10,000 parts. So, so, in a query like this one, where you have part number as well as Vendor ID, there is a possibility of using a composite index.

So, whether the Composite Index should be part number vendor id or vendor id part number, it should be part number vendor id. Why? Because when part number has a large number of possible values about 1000 whereas vendor id has very few possible values. So, if you can come to the specific part number, then your reduction in the number of records that you will have to look for the vendor id 1012 will be a lot more compared to if you had done vendor id,

part number where 1012 will have thousands of part numbers and you will have to look for 457 between them. So, this is, these are kind of the judgments which you will have to there is this is all based on the profile of your data and cannot be generalized as such.

(Refer Slide Time: 32:44)

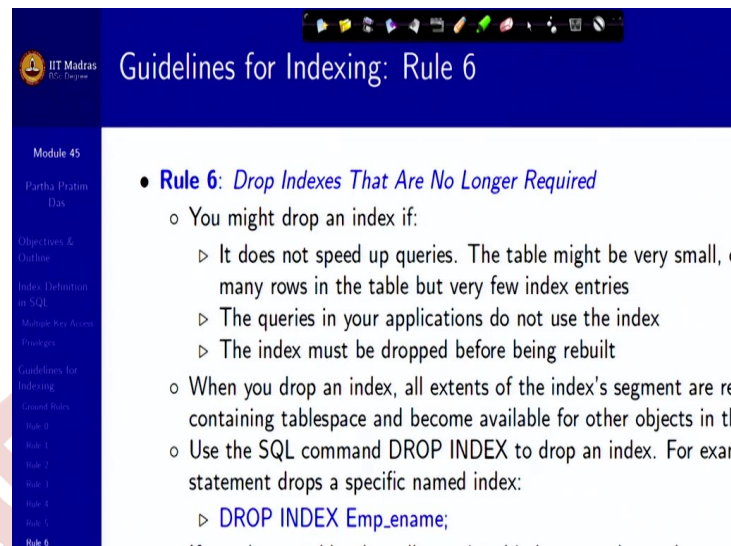
Guidelines for Indexing: Rule 5

Module 45
Partha Pratim Das

- **Rule 5: Gather Statistics to Make Index Usage More Accurate**
 - The database can use indexes more effectively when it has statistics about the tables involved in the queries
 - ▷ Gather statistics when the indexes are created by including the **COMPUTE STATISTICS** in the **CREATE INDEX** statement
 - ▷ As data is updated and the distribution of values changes, per the statistics by calling procedures like (in Oracle):
 - **DBMS_STATS.GATHER_TABLE_STATISTICS** and
 - **DBMS_STATS.GATHER_SCHEMA_STATISTICS**

So, you have to gather statistics to know about all this. So, you can I have shown how to use compute statistics. And you can also update the update and distribution of the value changes periodically by refreshing the statistics, you have to call these procedures if you are working in Oracle otherwise, you will have to see what the corresponding ones are.

(Refer Slide Time: 33:08)

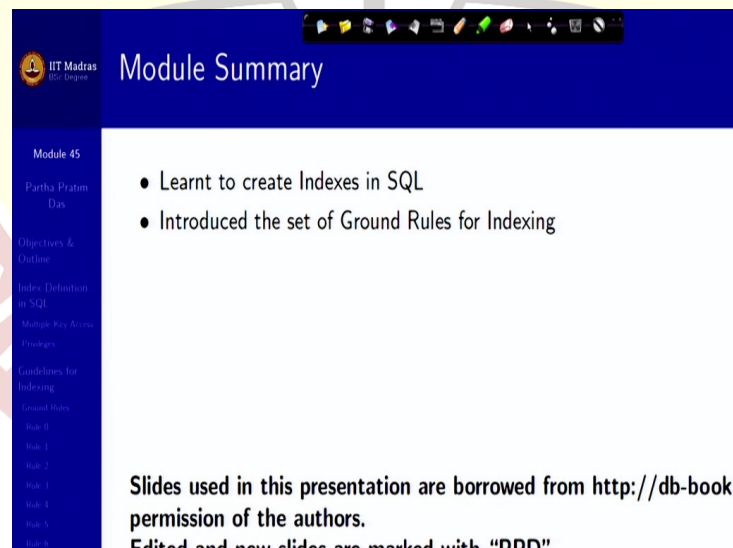


Guidelines for Indexing: Rule 6

- **Rule 6: Drop Indexes That Are No Longer Required**
 - You might drop an index if:
 - ▷ It does not speed up queries. The table might be very small, or have many rows in the table but very few index entries
 - ▷ The queries in your applications do not use the index
 - ▷ The index must be dropped before being rebuilt
 - When you drop an index, all extents of the index's segment are released, containing tablespace and become available for other objects in the tablespace
 - Use the SQL command DROP INDEX to drop an index. For example, the following statement drops a specific named index:
`DROP INDEX Emp_ename;`

And drop an index that is no longer required, it is quite obvious why you will have to drop that. And when you drop a table, then all indexes on that table are dropped. And if you do not want any index on a table, can drop all existing indexes by dropping, drop any index kind of thing. And you need the privilege for that.

(Refer Slide Time: 33:31)



Module Summary

- Learnt to create Indexes in SQL
- Introduced the set of Ground Rules for Indexing

Slides used in this presentation are borrowed from <http://db-book> permission of the authors.
Edited and new slides are marked with "DDN"

So, this brings us to the end of this module. We have now closed on our study on the indexing. We have learned how to create indexes of various types in SQL and what are the ground rules the guidelines for indexing. Thank you very much for your attention and we will meet in the next module.